Time spent on code: 15 hours
Time spent on slides: 5 hours

H02C8b Information Retrieval and Search Engines: Project Handout

Efe Ballar (r1072381)
March 15, 2026

1 Architecture

Figure 1 shows the full pipeline from user query to generated answer. The system consists of two phases: an **indexing** phase and a **retrieval and generation** phase.

Recipe dataset. Each recipe contains five fields: *name*, *tags*, *description*, *ingredients*, and *steps*.

doc_to_text. Concatenates the four selected fields (*name*, *tags*, *description*, *ingredients*) into a single string per document. The *steps* field is excluded (validated experimentally; see Section 3).

preprocess. Converts a raw string into a list of tokens via: (1) regex tokenization, (2) stopword/digit removal, and (3) lemmatization with `WordNetLemmatizer`.

Bigram model. A `gensim Phraser` model (Řehůřek and Sojka, 2010) trained on the tokenized corpus detects multi-word expressions by pointwise mutual information (PMI) with `min_count=5` and `threshold=3`. This allows compound terms such as *olive_oil* to be treated as single vocabulary entries.

build_vocab. Constructs the term vocabulary V from the corpus using document-frequency filtering: terms appearing in fewer than 5 documents or more than 50% of documents are excluded. The vocabulary is capped at 50 000 terms. IDF weights are computed as:

$$\text{idf}(t) = \log\left(\frac{N+1}{\text{df}(t)+1}\right) + 1$$

where N is the total number of documents and $\text{df}(t)$ is the number of documents containing term t. The $+1$ smoothing prevents zero IDF for terms appearing in all documents.

vectorize. Embeds a token list as a sparse TF-IDF vector. Term frequency is scaled as $\text{tf}(t) = 1 + \log(\text{count}(t))$, multiplied by the precomputed IDF weight, and the resulting vector is L2-normalized.

Document vector store. The N resulting sparse vectors are stored in memory as a list of dictionaries mapping vocabulary indices to TF-IDF weights.

vectorize_query. At query time, the user query undergoes the identical preprocessing pipeline and is vectorised using the *same* vocabulary and weights. Out-of-vocabulary queries produce a zero vector, which is handled explicitly.

dot_sparse. Computes cosine similarity between the query vector and each document vector via a sparse dot product. Since both vectors are L2-normalised, the dot product equals cosine similarity, making a separate normalization step unnecessary.

Top-k selection. Document scores are sorted in descending order and the top k documents are returned. The value of k is determined experimentally (see Section 4).

Prompt formatter. Combines the original query with the k retrieved recipes into a structured prompt using LLM's chat template. Unlike the indexing step, the *steps* field *is* included here.

Mistral-7B-Instruct. An instruction-tuned language model (Jiang et al., 2023) that generates a natural-language answer conditioned on the prompt. The model is loaded in 4-bit NF4 quantisation (Ouyang et al., 2022) to reduce memory requirements.

2 Term Vocabulary

Which terms enter V? The vocabulary V is derived entirely from the corpus rather than fixed in advance. Every token surviving the preprocessing pipeline is a candidate term. Candidates are then filtered: a term t is admitted to V only if

$$\texttt{min_df} \;\le\; \text{df}(t) \;\le\; \lfloor \texttt{max_df_ratio} \times N \rfloor$$

with $\texttt{min_df} = 5$ and $\texttt{max_df_ratio} = 0.5$. The lower bound removes terms that appear in too few documents, and the upper bound removes terms that appear in almost every recipe, such as *cook*, and *stir*.

Vocabulary size. The vocabulary size is *not* pre-determined. Fixing a hard cutoff may discard domain-relevant rare terms (e.g.*mirepoix*). Instead, the size emerges naturally from the document-frequency filter. A soft cap of $\texttt{max_vocab} = 50\,000$

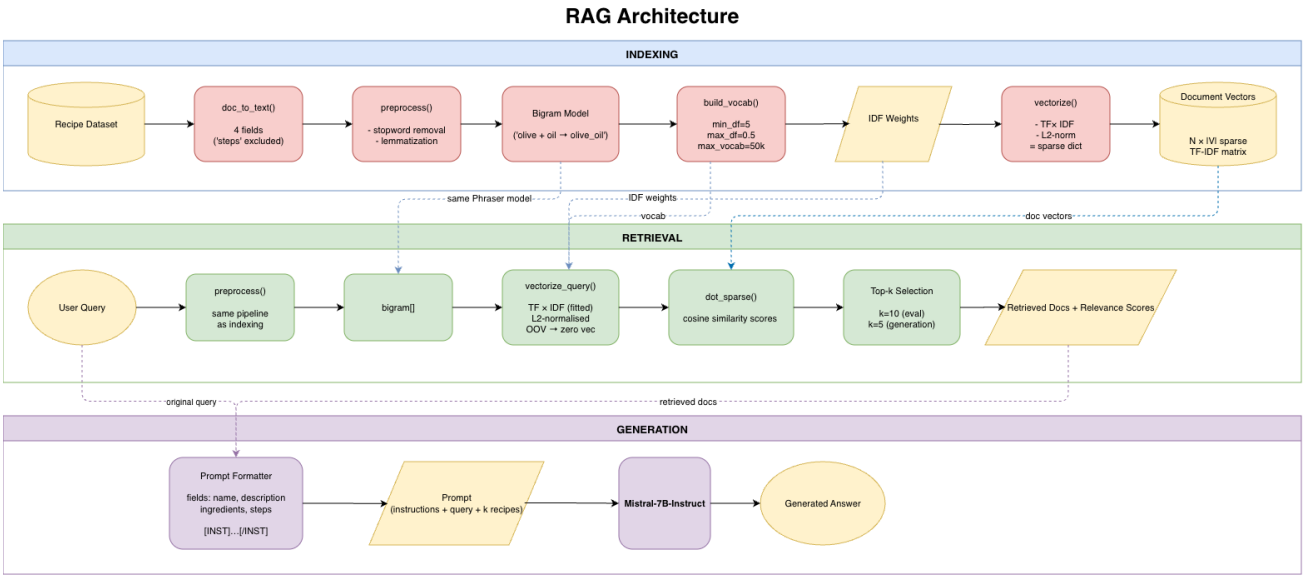


Figure 1 – Full architecture of the naive RAG pipeline. Dashed arrows indicate cross-phase dependencies.

is applied only as a safeguard against large vocabularies; in practice the filter alone produces a vocabulary well within this limit.

Multi-word expressions. Single-token representation fails for compound names such as *olive oil*, and *baking powder*: splitting them loses the compound meaning. To address this, a bigram model is trained on the tokenized corpus using `gensim Phrases` (Řehůřek and Sojka, 2010) with `min_count=5` and `threshold=3.0`. A bigram (w_1, w_2) is added to the vocabulary as a single token when the two words co-occur significantly more than expected by chance:

$$\frac{\text{count}(w_1 w_2) \cdot N}{\text{count}(w_1) \cdot \text{count}(w_2)} > \delta$$

where N is the total number of tokens in the corpus and $\delta = 3.0$ is the threshold. The same bigram model is applied to both documents and queries at vectorization time for consistency.

3 Document Embeddings

Field selection. Each recipe has five fields: *name*, *tags*, *description*, *ingredients*, and *steps*. To determine which fields to include in the document embeddings, an experiment using $k = 10$ has been ran, varying the field combination while keeping all other pipeline components fixed. Results are shown in Table 1; see `field_experiment.py` for the experiment code.

The largest single improvement comes from adding *description*, which nearly doubles MAP compared to *name + ingredients* alone. Adding *tags* provides a further consistent improvement across all combinations, as tags such as *vegetarian* directly match query intent words. Including *steps* hurts MAP despite increasing recall slightly, because procedural language (*add*, *stir*) is uniformly distributed across recipes and dilutes the discriminative signal. The final field combination used in the pipeline is therefore **name + ingredients + description + tags**. You may notice that these scores are quite low, this is expected given the nature of TF-IDF: many gold queries require semantic reasoning (e.g. "What temperature should I pre-heat my oven to?") rather than lexical matching, which fundamentally limits retrieval quality. Finally, note that *steps* is excluded from IR indexing but included when formatting recipes for the language model, since the procedural detail is needed to answer queries.

Query vectorization. The query string undergoes the same preprocessing pipeline as the documents: regex tokenisation, stopword removal, digit removal, lemmatisation, and bigram expansion. This ensures queries and documents reside in the same vector space. The resulting token list is then embedded using the same TF-IDF formula as the documents. For each term t in the query:

$$\begin{aligned}\vec{q}_t &= \text{tf}(t) \times \text{idf}(t) \\ &= \big(1 + \log \text{count}(t)\big) \times \left(\log \frac{N+1}{\text{df}(t)+1} + 1\right)\end{aligned}$$

The resulting vector is L2-normalized, making it directly comparable to the normalized document vectors via dot product.

Table 1 – Retrieval performance by field combination ($k = 10$, no threshold). Best MAP highlighted.

Fields	macro-P	macro-R	macro-F1	MAP
name	0.054	0.071	0.054	0.038
name + tags	0.046	0.069	0.048	0.042
name + ingredients	0.046	0.070	0.050	0.030
name + ingredients + tags	0.059	0.076	0.057	0.037
name + ingredients + description	0.076	0.088	0.070	0.037
name + ingredients + description + tags	**0.081**	**0.096**	**0.075**	**0.045**
name + ingredients + description + tags + steps	0.076	0.087	0.070	0.033

Out-of-vocabulary queries. If every term in the query is absent from V, the query vector is the zero vector. This is detected explicitly: a zero vector produces undefined cosine similarity, so the system returns an empty result set and reports that no matching terms were found. For partially out-of-vocabulary queries, known terms still contribute to the query vector and retrieval degrades gracefully, which is a natural property of TF-IDF.

4 Retrieval

Similarity operator. I chose **cosine similarity** as the nearest-neighbour operator. Since both document and query vectors are L2-normalised during vectorisation, cosine similarity reduces to a sparse dot product:

$$\text{sim}(\vec{q}, \vec{d_i}) = \vec{q} \cdot \vec{d_i} = \sum_{t \in V} \vec{q_t} \cdot \vec{d}_{i,t}$$

Cosine similarity is the standard choice for TF-IDF retrieval (Manning et al., 2008) because it is invariant to document length, so a short recipe and a long recipe with similar content score equally. This choice does not require experimental validation as it is a well-established property.

Determining k. k was chosen empirically via a sensitivity experiment run over $k \in \{3, 5, 10, 20, 50\}$ on the gold query set (see `evaluate.py`). I also ran a threshold sensitivity experiment to test if filtering retrieved documents by minimum cosine score improves results. Table 2 and Table 3 show the results.

The threshold experiment shows that all thresholds below 0.25 produce identical results, meaning every top-3 document already scores above 0.25 naturally, **k is the binding constraint, not the threshold**. I therefore set threshold to 0 and focus on k. The k sensitivity table shows the classic precision/recall tradeoff: smaller k yields higher precision, larger k yields higher recall. I select **$\mathbf{k = 10}$** as the final value since MAP better captures ranking quality, and $k = 10$ maximises MAP. However note that for the LLM's generation step, $k = 5$ is used to keep the prompt within the language model's context window.

Table 2 – Threshold sensitivity ($k = 3$). Thresholds below 0.25 are identical, indicating k is the binding constraint.

Threshold	macro-P	macro-R	macro-F1
0.05	0.163	0.083	0.095
0.10	0.163	0.083	0.095
0.15	0.163	0.083	0.095
0.20	0.163	0.083	0.095
0.25	0.155	0.077	0.088
0.30	0.167	0.077	0.089
0.35	0.142	0.065	0.073
0.40	0.073	0.038	0.046

Table 3 – k sensitivity (threshold=0). $k = 3$ maximises macro-F1; $k = 10$ maximises MAP.

k	macro-P	macro-R	macro-F1
3	0.163	0.083	0.095
5	0.102	0.083	0.077
10	0.081	0.115	0.080
20	0.055	0.148	0.069
50	0.031	0.178	0.048

Macro and micro averaged metrics. All metrics are computed on the given 41 answerable gold queries (6 queries with no relevant documents are excluded). Full results are in Table 4.

Table 4 – Retrieval evaluation results ($k = 10$, threshold=0).

Metric	Macro	Micro
Precision	0.0805	0.0805
Recall	0.1148	0.0570
F1	0.0798	0.0667

Macro averages compute each metric per query and

then average across queries, treating every query equally. Micro averages aggregate true positives, and relevant documents before computing metrics, giving more weight to queries with many relevant documents. The identical macro and micro precision values are a consequence of fixed $k = 10$: every query retrieves exactly 10 documents, so $\sum |\hat{D}_j| = 10|Q|$ in both formulations.

Mean Average Precision (MAP). MAP is defined as:

$$\text{MAP} = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \text{AP}(q_j)$$

where $|Q| = 41$ is the number of answerable queries and $\text{AP}(q_j)$ is the average precision for query q_j:

$$\text{AP}(q_j) = \frac{1}{|R_j|} \sum_{k=1}^{K} P@k \cdot \text{rel}(k)$$

Here $|R_j|$ is the total number of relevant documents for query q_j; K is the total number of retrieved documents; $P@k$ is the precision computed considering only the first k documents in the ranked list, i.e. $P@k = \frac{1}{k} \sum_{i=1}^{k} \text{rel}(i)$; and $\text{rel}(k) \in \{0, 1\}$ indicates whether the document at rank k is relevant. The phrase *at k* means: restrict attention to the top k retrieved documents. System achieves MAP = **0.0618**.

5 Qualitative Analysis (TF-IDF)

I ran five queries through the retrieval pipeline ($k = 3$, threshold=0) to identify failures of TF-IDF. Three results per query are sufficient to illustrate each drawback.

Drawback 1: No understanding of query. *Query:* `vegetarian lasagna but easy`
Retrieved: pumpkin pine nut lasagna (0.555), one step lasagna (0.55), easy vegetable lasagna (0.51)

TF-IDF matched on *lasagna* and *easy* correctly, but has no understanding of what *vegetarian* means as a constraint. The first two results may contain meat, TF-IDF cannot reason about dietary requirements.

Drawback 2: No tolerance for spelling errors. *Query:* `spageti and meatballs?`
Retrieved: easy meatballs (0.6), party meatball sauce (0.57), homemade fabulous meatballs (0.53)

The misspelling *spageti* is out-of-vocabulary and silently dropped. The entire query is resolved by *meatball* alone, skewing results toward meatball dishes and missing spaghetti entirely.

Drawback 3: Cannot handle units, numbers. *Query:* `lunch baked at 200 °C`
Retrieved: baked corn dogs (0.33), cheesy baked egg (0.31), fish stick batter (0.3)

The degree symbol `°` is removed by the tokeniser, `200` is filtered as a digit, and *lunch* is too common to discriminate. The remaining token *baked* matches almost anything oven-related. The results are completely irrelevant.

Drawback 4: Vocabulary mismatch with relevant documents. *Query:* `shrimp tacos recipe for tonight`
Retrieved: simple shrimp tacos (0.37), u peel em shrimp (0.34), firecracker shrimp tacos (0.32)

The second result, *u peel em shrimp*, has no relation to tacos. It ranks highly because *shrimp* has high TF-IDF weight and dominates the score. TF-IDF cannot weigh multiple query terms jointly, a document matching only one important term can outscore others.

Drawback 5: Cannot handle reasoning-based queries. *Query:* `cajun style gumbo with an easy roux`
Retrieved: gumbo base aka roux (0.41), double roux cajun seafood gumbo (0.4), cajun style chicken gumbo (0.39)

The results are reasonable, but none directly answers the query, which is a gumbo recipe that specifically includes an easy roux method. TF-IDF retrieves documents containing the right words but cannot reason if a recipe satisfies the implied constraint of simplicity.

6 Prompt Design

Chat template. The model used is Mistral-7B-Instruct-v0.2 (Jiang et al., 2023). Prompts must follow the `[INST]...[/INST]` chat template, with `<s>` marking the beginning of the sequence.

Fields used in the prompt. Each retrieved recipe is formatted with four fields: *name*, *description*, *ingredients*, and *steps*. Notably, *steps* is included here despite being excluded from IR indexing, because the language model needs procedural detail to answer reasoning queries. Tags are excluded from the prompt since they add little natural-language context.

Prompt A. Gives bare instructions:

Prompt A

```
<s>[INST] Answer the following question
using only the provided recipes.
Question: {query}
Recipes: Recipe 1: {name}
Description: {description}
Ingredients: {ingredients} Steps:
{steps}
... (repeated for k recipes)
Answer: [/INST]
```

Prompt B. Assigns a role, prohibits use of external knowledge, and requests a concise answer:

Prompt B

```
<s>[INST] You are a culinary assistant.
You will be given a question and a set
of relevant recipes retrieved from
a cookbook database. Your task is
to answer the question by reasoning
across the provided recipes. Do
not use any knowledge outside of the
provided recipes. If the recipes
do not contain enough information to
answer the question, say so explicitly.
Question: {query}
Retrieved recipes: Recipe 1:
{name} Description: {description}
Ingredients: {ingredients} Steps:
{steps}
... (repeated for k recipes)
Based only on the recipes above, answer
the question concisely. [/INST]
```

Comparison. Table 5 shows example outputs for the same query under both prompts.

Table 5 – Prompt A vs. Prompt B on query: *What is the most important ingredient in borscht?*

Prompt	Response
A	The most important ingredient in borscht varies among different recipes. In Recipe 1, it is the farmer sausage or smoked ham hock. In Recipe 2, it is the beets. In Recipe 3, it is the combination of chicken, vegetables, and beans. In Recipe 4, it is the beetroot. In Recipe 5, it is the red onions. Therefore, there is no single most important ingredient.
B	Based on the provided recipes, one common ingredient among all recipes is beets, which plays a significant role in giving borscht its distinctive color and flavor. Therefore, beets can be considered an essential ingredient in making a traditional borscht.

Prompt A enumerates each recipe individually, however the role assignment and explicit instruction to reason across recipes produces meaningfully better output, making Prompt B the template used for all subsequent experiments.

7 Qualitative Analysis (LM)

All experiments in this section are run via `main.py` using Prompt B as the default template and $k = 5$ retrieved recipes, k is kept smaller than the evaluation value of 10 to avoid exceeding the language model's context window.

Does the LM understand the RAG task? To test this, I supplied the model with an irrelevant context, and asked a cooking question. The model understands the RAG task:

irrelevant context

Query: `What temperature should I bake chicken Wellington at?`
Context: `Richard Gary Brautigan (January 30, 1935 - c. September 16, 1984) was an American novelist, ...`
Response: `I'm unable to provide an answer to your question about baking temperature for chicken Wellington based on the provided context. The context only mentions information about the author Richard Gary Brautigan and his literary works.`

Reasoning across recipes. Table 6 shows five queries (with recipes themselves are not an answer) whose answers require reasoning across the retrieved recipes.

Each answer requires the model to synthesize information across multiple retrieved recipes rather than returning any single recipe as an answer.

Grounding verification. To verify the model does not use its own knowledge, I ran the same query with and without RAG context (Experiment 4, `main.py`):

grounding check

Query: `What is a turducken?`
Retrieved: `turducken thanksgiving sides, turducken roulade, turducken chicken duck turkey, turducken, umami stuffing`
With RAG context: `A turducken is a dish consisting of a deboned turkey stuffed with a deboned duck, which in turn is stuffed with a deboned chicken. The deboning process is time-consuming and may be done the day before to save time. The turducken is then seasoned, stuffed with prepared stuffing, and slow-cooked. The recipes provide detailed instructions for deboning and assembling a turducken, as well as preparing Cajun-inspired stuffing and rice dressing.`
Without context (bare LM): `A turducken is a culinary dish that consists of a deboned turkey stuffed with a duck, which in turn is stuffed with a chicken. It is a combination of three poultry dishes in one, and is considered an intricate and indulgent holiday meal.`

RAG answer contains specific details absent from the bare LM response, which gives a generic description with no procedural detail, confirming the model grounds its answers in the supplied context.

Score sensitivity. Tested if including retrieval scores in the prompt changes the model's output (Experiment 5, `main.py`). A third prompt, Prompt B+scores, annotates each recipe with its cosine similarity score:

score sensitivity

Query: `What temperature should I bake chicken Wellington at?`
Scores: `[0.362, 0.341, 0.315, 0.302, 0.291]`
Without scores: `The chicken Wellington recipe in Recipe 1 states to preheat the oven to 180c / 160c fan forced.`
With scores: `The recipe with the highest relevance score, Recipe 1, specifies that the oven should be preheated to 180c / 160c fan forced for baking chicken Wellington.`

Including retrieval scores has no significant effect on answer quality for this system.

Table 6 – Five reasoning queries with retrieved recipes and model answers (Experiment 3, `main.py`).

Query	**Retrieved**	**Answer**
What temperature should I pre-heat my oven to when making chicken quesadillas?	quesadilla combos, bbq quesadillas, popovers pre 1940, chicken jalapeno quesadillas nuwave oven, jillyray cheese quesadillas salsa	According to Recipe 2 and Recipe 4, the oven should be pre-heated to 425 degrees Fahrenheit.
What is the most important ingredient in borscht?	somma summer krut weed borscht, borscht ii, chicken vegetable borscht, simple borscht, red onion borscht	Based on the provided recipes, beets are a common ingredient appearing in most recipes and can be considered the most important ingredient in borscht.
Is beef Wellington the only kind of meaty Wellington?	pork wellington, beef wellington two, individual pork wellingtons, beef wellington ramsay style, individual beef scallop wellingtons rachael ray	Beef Wellington is not the only kind. Pork Wellington and a sea scallop variation also exist in the provided recipes.
Where does shakshuka come from?	shakshuka yemani basted eggs, shakshuka swiss chard, shannon shakshuka, shakshuka, ragu shuka ragu	Shakshuka is a dish that originated from North Africa and the Middle East. The recipes do not provide specific information about its origins.
What are the three essential ingredients in a baked Alaska?	heavenly baked alaska, simple baked alaska, baked alaska salmon fillet mustard sauce, chocolate baked alaska, quick baked alaska	The three essential ingredients are ice cream, a cake base (sponge cake or jelly roll), and meringue made from egg whites and sugar.

Bibliography

Jiang, Albert Q., Sablayrolles, Alexandre, Mensch, Arthur, Bamford, Chris, Chaplot, Devendra Singh, Casas, Diego de las, Bressand, Florian, Lengyel, Gianna, Lample, Guillaume, Saulnier, Lucile, Lavaud, Lélio Renard, Lachaux, Marie-Anne, Stock, Pierre, Scao, Teven Le, Lavril, Thibaut, Wang, Thomas, Lacroix, Timothée, and Sayed, William El (Oct. 2023). *Mistral 7B*. arXiv:2310.06825 [cs]. DOI: 10.48550/arXiv.2310.06825. URL: http://arxiv.org/abs/2310.06825 (visited on 2024-02-20).

Manning, Christopher D., Raghavan, Prabhakar, and Schütze, Hinrich (2008). *Introduction to Information Retrieval*. Cambridge University Press. URL: https://nlp.stanford.edu/IR-book/.

Ouyang, Long, Wu, Jeff, Jiang, Xu, Almeida, Diogo, Wainwright, Carroll L., Mishkin, Pamela, Zhang, Chong, Agarwal, Sandhini, Slama, Katarina, Ray, Alex, Schulman, John, Hilton, Jacob, Kelton, Fraser, Miller, Luke, Simens, Maddie, Askell, Amanda, Welinder, Peter, Christiano, Paul, Leike, Jan, and Lowe, Ryan (Mar. 2022). *Training language models to follow instructions with human feedback*. arXiv:2203.02155 [cs]. DOI: 10.48550/arXiv.2203.02155. URL: https://cdn.openai.com/papers/Training_language_models_to_follow_instructions_with_human_feedback.pdf (visited on 2023-02-04).

Řehůřek, Radim and Sojka, Petr (2010). "Software Framework for Topic Modelling with Large Corpora". In: *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. ELRA, pp. 45–50.